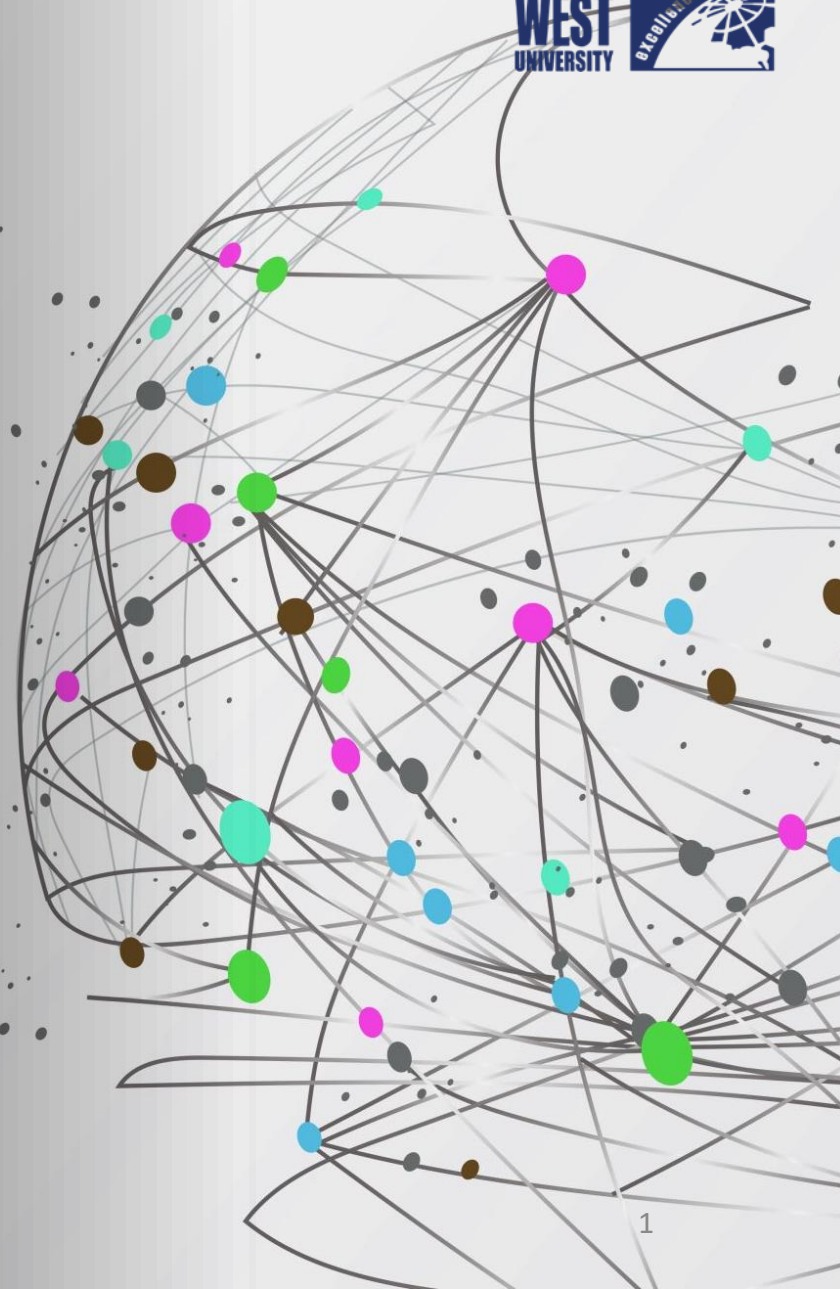
CSE400C · CAPSTONE PROJECT · SUMMER 2026

Automated Fabrics Defect Detection

Using Computer Vision and Deep Learning

A lightweight YOLO-based object detection framework for automated garment quality inspection



EAST WEST UNIVERSITY

Presented By

Sumaiya Hoque Riyan

(2022-1-60-111)

Md. Momtajur Rahman

(2022-1-60-370)

Jahir Hasan Biddut

(2022-1-60-096)

Md. Nahid Hasan

(2022-1-60-221)

Supervised By

Md Sabbir Hossain

Lecturer

Department of Computer Science & Engineering

East West University

Table Of Content

What We'll Cover

01 Introduction

02 Objectives & Literature Review

03 Methodology

04 Model Training

05 Experimental Results

06 Performance Comparison

07 FabDetector Web App

08 System Design

09 Discussion

10 Conclusion & Future Work

Why automate garment inspection?

- Small surface defects (holes, knots/slubs, stains, yarn faults) can make an otherwise finished garment unsellable — especially for export-grade buyers.
- Manual inspection is still the norm in most factories, but human accuracy drops with fatigue, poor lighting, and repetitive fabric patterns.
- Computer vision + deep learning can inspect fabric consistently, at speed, and provide visual evidence for every decision — the same idea behind FabDetector.

MORE THAN 70

documented fabric defect types exist in the literature — holes, stains, broken threads, bars, broken weft, mis-weaves.

- **Consistent & Precise**
  Objective, repeatable results 24/7
- **Rapid & Scalable**
  Inspects at production-line speed
- **Actionable Insight**
  Classifies and localizes, not just flags

Problem statement

Finding a defect isn't enough — the system must say where it is and what type it is, under difficult real-world conditions

Detecting garment defects is not just a yes/no task — the system must locate where a defect is on the fabric and identify what type it is. Defects vary widely in size, shape, color, and texture, and often blend into complex, repeating fabric patterns.

1

Visual variability

Defects vary hugely in size, shape, color, and texture — some barely visible against the weave.

2

Complex fabric patterns

Repeated designs can be mistaken for defects, or hide real ones inside the pattern.

3

Class imbalance

Rare defect types are underrepresented, so the model learns them poorly.

4

Limited annotated data

Labeling is slow and costly, restricting how much training data is available.

Goal: an automated system that detects, distinguishes defect-free vs. defective, localizes the defect, and runs in near real time for industrial use.

Research objectives & questions

OBJECTIVES

- Develop an automated garment defect detection system using deep learning.
- Train and compare YOLOv8s, YOLOv9s, YOLOv11s, and YOLO26s on the same dataset.
- Design a complete preprocessing → training → evaluation pipeline.
- Identify the most accurate, efficient model for industrial deployment.

KEY RESEARCH QUESTIONS

1. How can a deep-learning object detector automatically identify and localize garment defects?
2. What performance can a lightweight YOLO model reach in precision, recall, and mAP on this dataset?
3. Which defect categories are easiest / hardest to detect, and why?
4. How does dataset class imbalance affect detection performance?

09 · OBJECTIVES

Manual QC vs. FabDetector

Manual inspection scales with headcount and hours — FabDetector scales with compute.

MANUAL QC (CURRENT PRACTICE)

TIME PER PIECE
1.5–3 min (up to 5 min for complex knits)

STAFFING
~1 inspector per 8–12 operators, plus 4–8 end-line checkers

CONSISTENCY
Varies with fatigue, lighting, experience

COST STRUCTURE
Recurring — scales with headcount & shift hours

FABDETECTOR (AUTOMATED)

TIME PER PIECE
Near real-time — sub-second inference per image

STAFFING
No added per-line inspector required

CONSISTENCY
98.56% test accuracy, uniform on every run

COST STRUCTURE
~Tk 3.14 lakh first-year (lean deployment)

Related Work at a Glance

Study	Method	Strength	Limitation
Rasheed et al. survey	Faster R-CNN, YOLO-LFD, DCGAN	Strong localization & learned features	High data demand, limited public datasets
Deep-GD (Mathew & Brintha)	Bilateral filter + SENet + RDF-BO	High accuracy (97.16%)	Complex pipeline, not real-time / lightweight
IJNRD consumer study	CNN + GUI quality check	Consumer-facing, simple	Weak industrial grounding, less rigorous
This capstone	YOLOv8s / v9s / v11s / v26s	End-to-end, fast, localizes + classifies	Sensitive to class imbalance & fine textures

Across 15+ reviewed studies, the field trends toward lightweight, single-stage detectors — motivating this project's focus on comparing modern YOLO variants for real-time garment inspection.

End-to-End Methodology

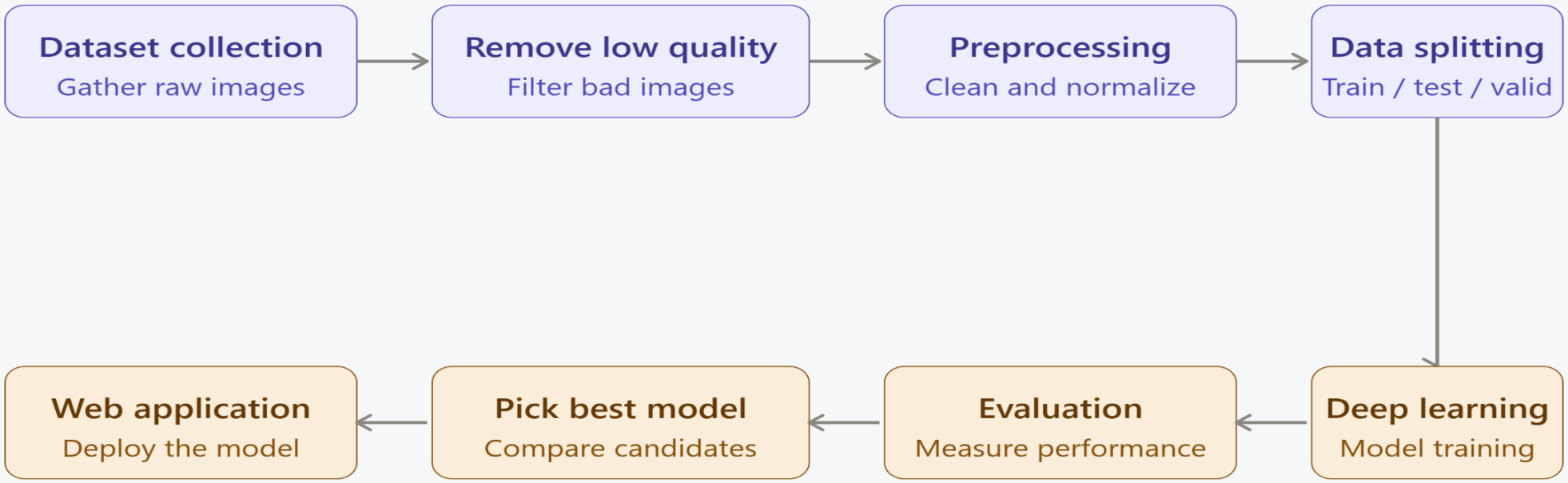


Figure2:Workflow of our automation project

Data Collection

Field data captured on-site and prepared for model training

1 Manual Field Collection

Raw image data manually collected through repeated on-site field visits to APT Sweater Limited (APT Garments), Gazipur — a 100% export-oriented sweater manufacturing factory.



On-site field visit — yarn winding floor, APT Sweater Ltd.

2 Manual Annotation in Roboflow

All 1887 image manually annotated with bounding boxes in Roboflow, labeling objects of interest to build a clean, ground-truth dataset for training.



Field Work
Multiple site visits

Manual
Bounding box labels

Roboflow
Annotation platform

Dataset overview

Fabric Class	Original Instances	Train (Aug.) Instances
Defect-Free	470	470
Spot	443	443
Hole	367	367
Knot Slub	308	≈ 1,232
Yarn	299	≈ 1,196
Total	**1,887**	**≈ 3,708**



Figure2 — sample training images across all five classes

Four lightweight YOLO architectures

	YOLOv8s	YOLOv9s	YOLOv11s	YOLO26s
Release year	2023	2024	2024	2025
Backbone	CSP + C2f	GELAN	C3k2 / C2PSA	Optimized CSP
Detection head	Anchor-free, decoupled	Anchor-free, decoupled	Anchor-free, decoupled	Anchor-free, decoupled
Main strength	Speed–accuracy balance	Gradient propagation	Improved localization	Lightweight, fast convergence

All four share an anchor-free, decoupled detection head and multi-scale P3/P4/P5 feature pyramid — the differences that matter most are backbone design and how efficiently each aggregates features.

Model architecture

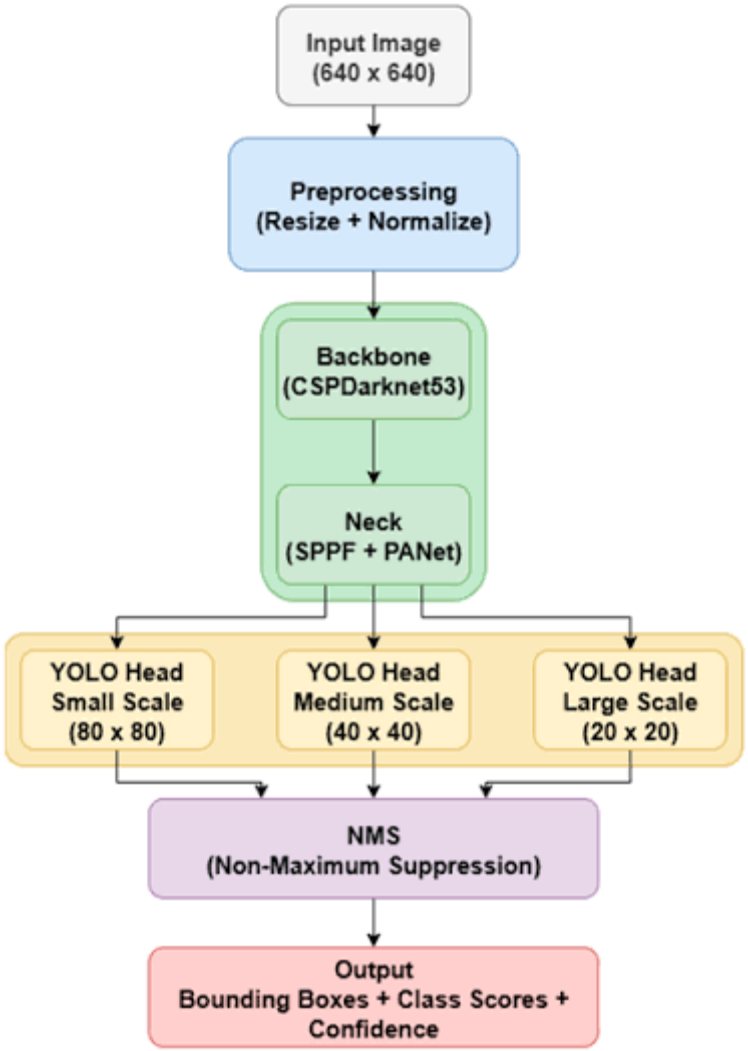


Figure1. illustrates the model architecture.

Backbone — CSPDarknet-style network extracts hierarchical visual features.

Neck — SPPF + PANet aggregate features across multiple scales.

Head — Three detection heads predict boxes at small / medium / large scale.

NMS — Non-maximum suppression yields final boxes, classes & confidence.

Training setup

MODEL & TRANSFER LEARNING

YOLOv, pretrained weights (yolov8s.pt) fine-tuned end-to-end on the fabric defect dataset

INPUT SIZE

640 × 640 px (imgsz = 640)

DATA SPLIT

Train 1,614 → 3,250 after augmentation · Validation 480 · Test 322

OPTIMIZER & LR SCHEDULE

AdamW · lr0 = 1e-3 with cosine decay (lrf = 0.01) · 5-epoch warmup

TRAINING BUDGET

Up to 400 epochs on **Kaggle GPU** · early stop (patience = 20) · checkpoint saved every 25 epochs, best kept

BUILT-IN AUGMENTATION

Mosaic (1.0), mixup (0.1), copy-paste (0.1), flips, HSV & rotation jitter — layered on top of the offline Albumentations pass

REGULARIZATION & PRECISION

Label smoothing (0.1) to offset annotation noise · automatic mixed precision (AMP) for faster training

METRICS TRACKED

Precision, Recall, F1-score, mAP@50, mAP@50:95

YOLOv8s — detection performance

Train 70% , Test 10%,Validation 20%

Test Accuracy: 98.56%

Best Epoch 372

	Conf Threshold	Precision	Recall	F1	mAP@50
0	0.2000	0.9557	0.9734	0.9645	0.9812
1	0.2500	0.9557	0.9734	0.9645	0.9812
2	0.3000	0.9532	0.9776	0.9652	0.9717
3	0.3500	0.9621	0.9687	0.9653	0.9642
4	0.4000	0.9693	0.9531	0.9611	0.9470
5	0.4500	0.9766	0.9333	0.9545	0.9318
6	0.5000	0.9900	0.9189	0.9531	0.9160

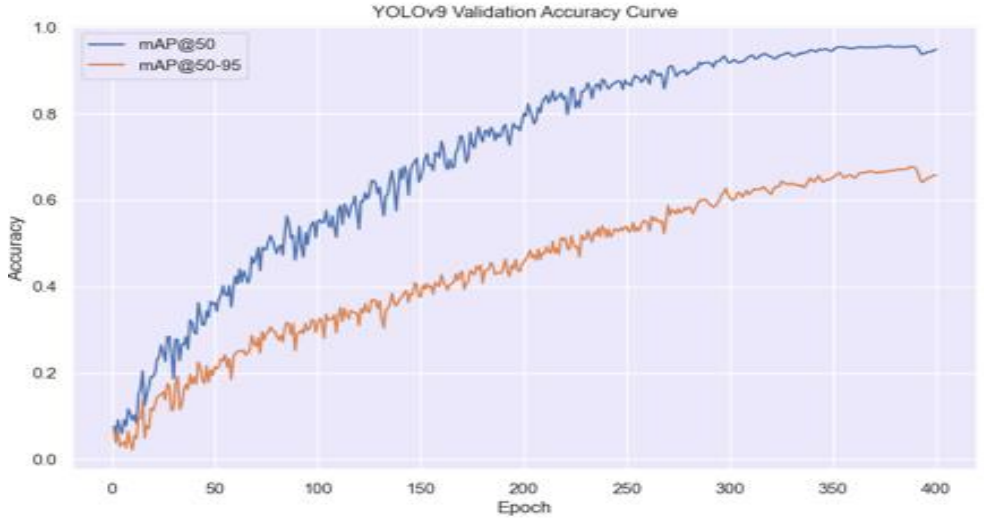


Figure 3. YOLOv8s Training and Validation Accuracy.

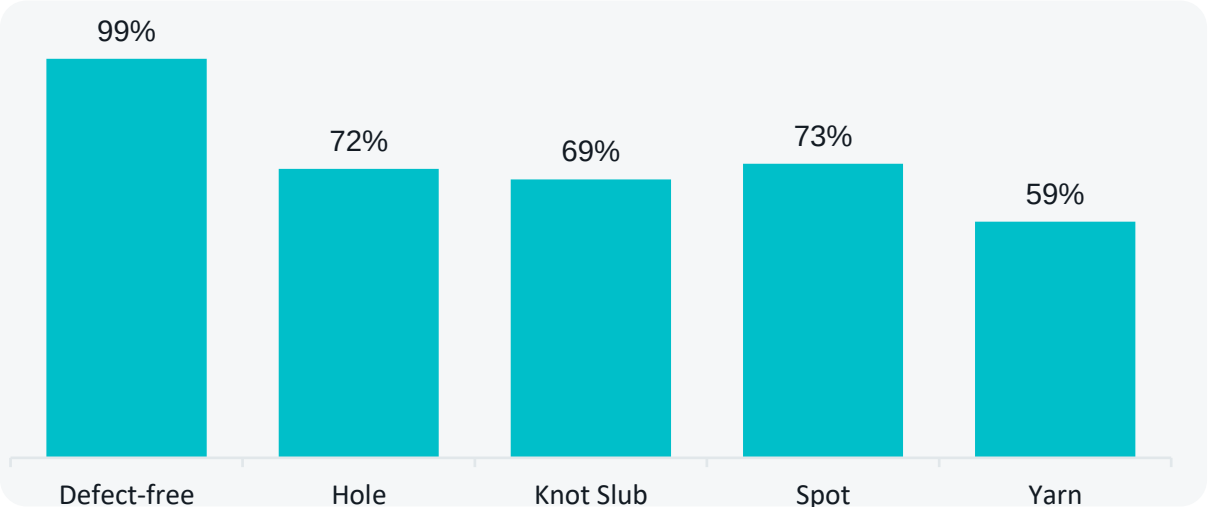


Figure 4. YOLOv8s per class mAP@50

YOLOv9s — detection performance

Train 70% , Test 10%,Validation 20%

Test Accuracy: 95.73%

Best Epoch 377

	Conf Threshold	Precision	Recall	F1	mAP@50
0	0.2000	0.9434	0.9142	0.9286	0.9390
1	0.2500	0.9432	0.9142	0.9285	0.9277
2	0.3000	0.9398	0.9147	0.9271	0.9072
3	0.3500	0.9499	0.9088	0.9289	0.8999
4	0.4000	0.9602	0.8870	0.9222	0.8797
5	0.4500	0.9596	0.8598	0.9070	0.8534
6	0.5000	0.9729	0.8296	0.8956	0.8249

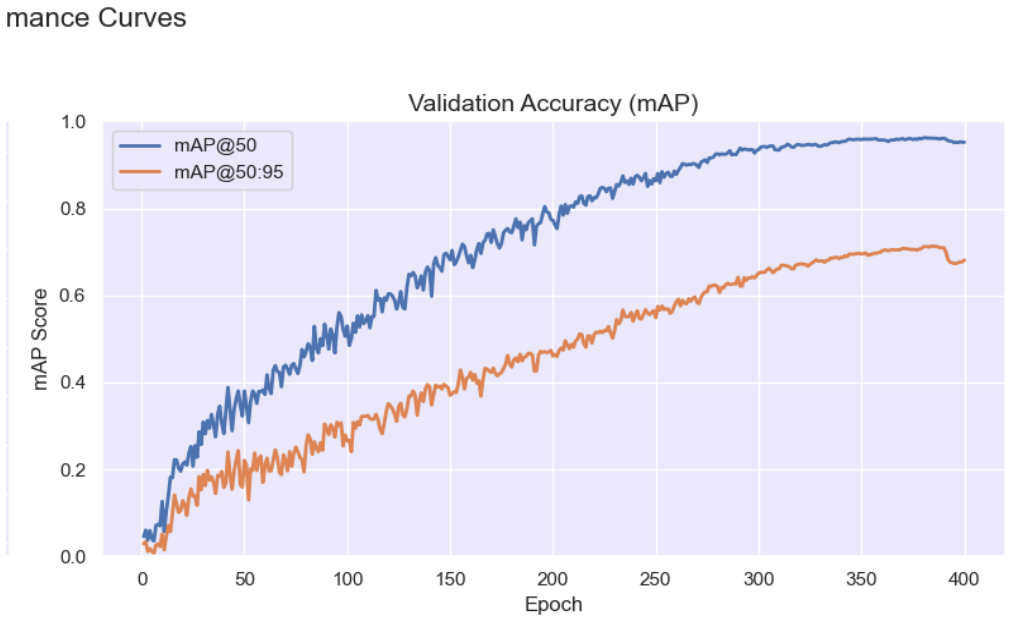


Figure 5. YOLOv9s Training and Validation Accuracy.

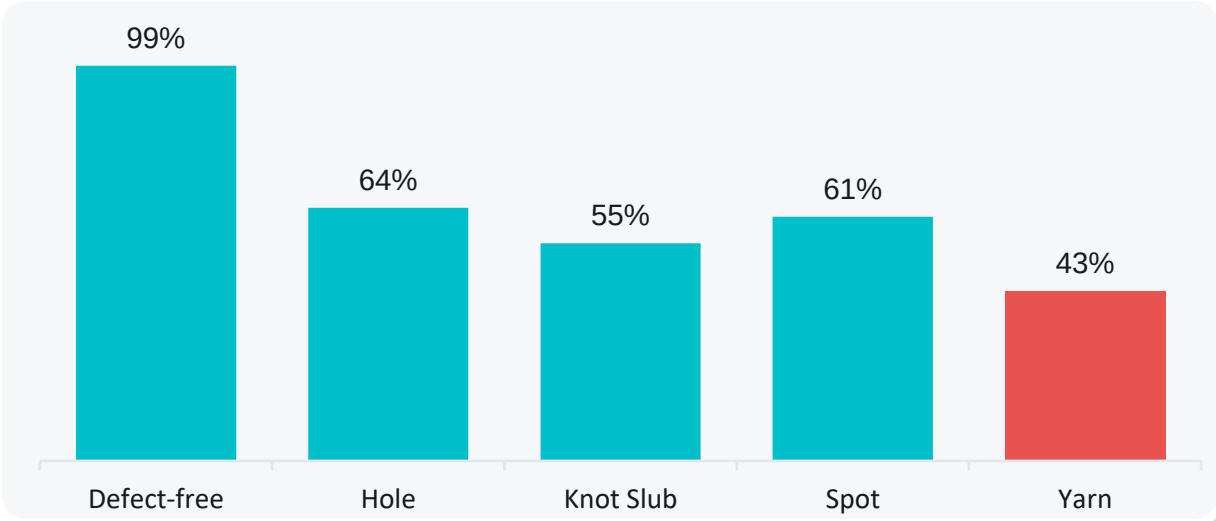


Figure 6. YOLOv9s per class mAP@50

YOLOv11s — detection performance

Train 70% , Test 10%,Validation 20%

Test Accuracy: 96.26 %

Best Epoch 381

	Conf Threshold	Precision	Recall	F1	mAP@50
0	0.2000	0.9194	0.9456	0.9323	0.9563
1	0.2500	0.9088	0.9633	0.9353	0.9484
2	0.3000	0.9255	0.9404	0.9329	0.9278
3	0.3500	0.9422	0.9176	0.9297	0.9086
4	0.4000	0.9568	0.9044	0.9298	0.8972
5	0.4500	0.9652	0.8798	0.9205	0.8745
6	0.5000	0.9686	0.8574	0.9096	0.8516

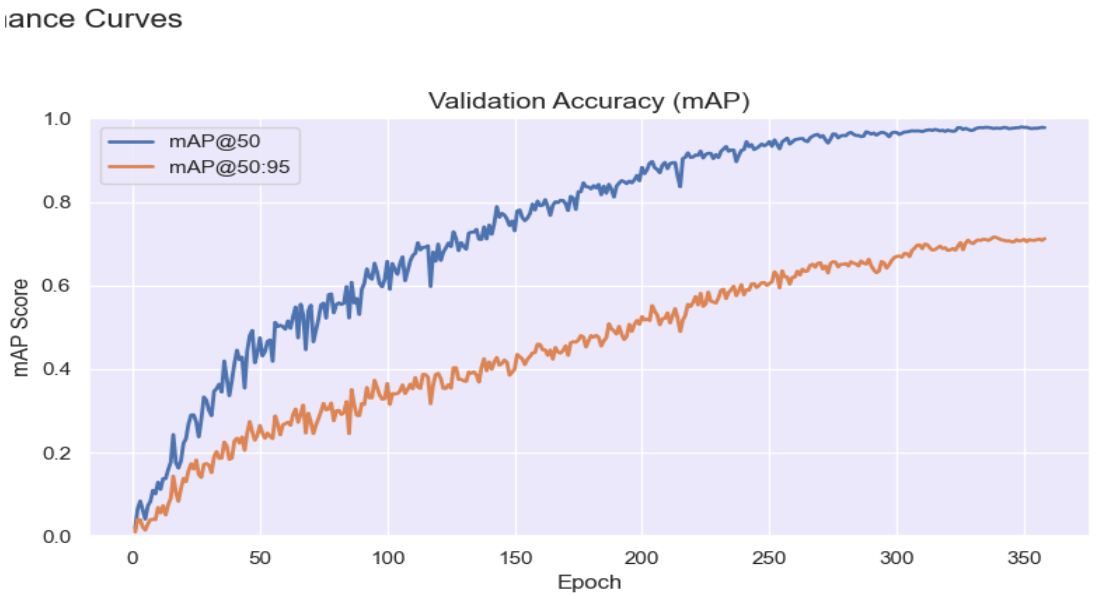


Figure 7. YOLOv11s Training and Validation Accuracy.

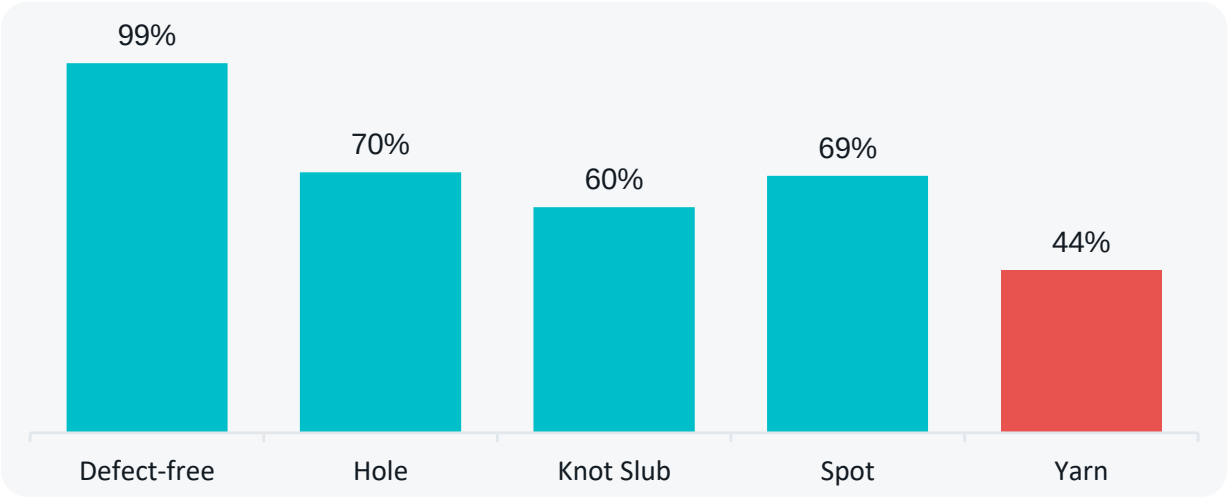


Figure 8. YOLOv11s per class mAP@50.

YOLO26s — detection performance

Train 70% , Test 10%,Validation 20%

Test Accuracy: 97.95%

Best Epoch 349

	Conf Threshold	Precision	Recall	F1	mAP@50
0	0.2000	0.9324	0.9312	0.9318	0.9571
1	0.2500	0.9324	0.9312	0.9318	0.9434
2	0.3000	0.9220	0.9412	0.9315	0.9329
3	0.3500	0.9381	0.9230	0.9305	0.9165
4	0.4000	0.9497	0.9118	0.9304	0.9072
5	0.4500	0.9580	0.8945	0.9252	0.8903
6	0.5000	0.9756	0.8654	0.9172	0.8631

ıance Curves

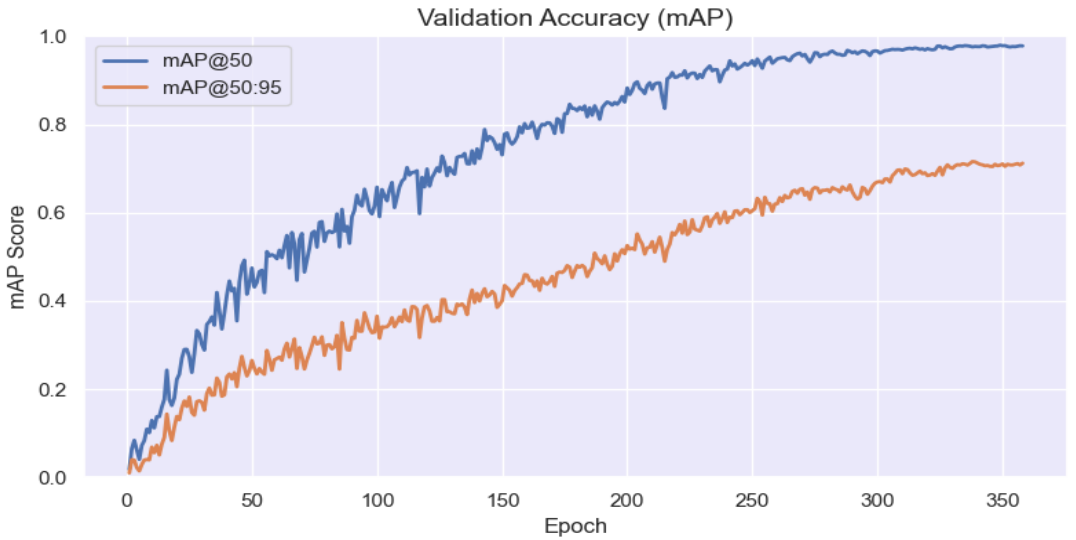


Figure 9. YOLO26s Training and Validation Accuracy.

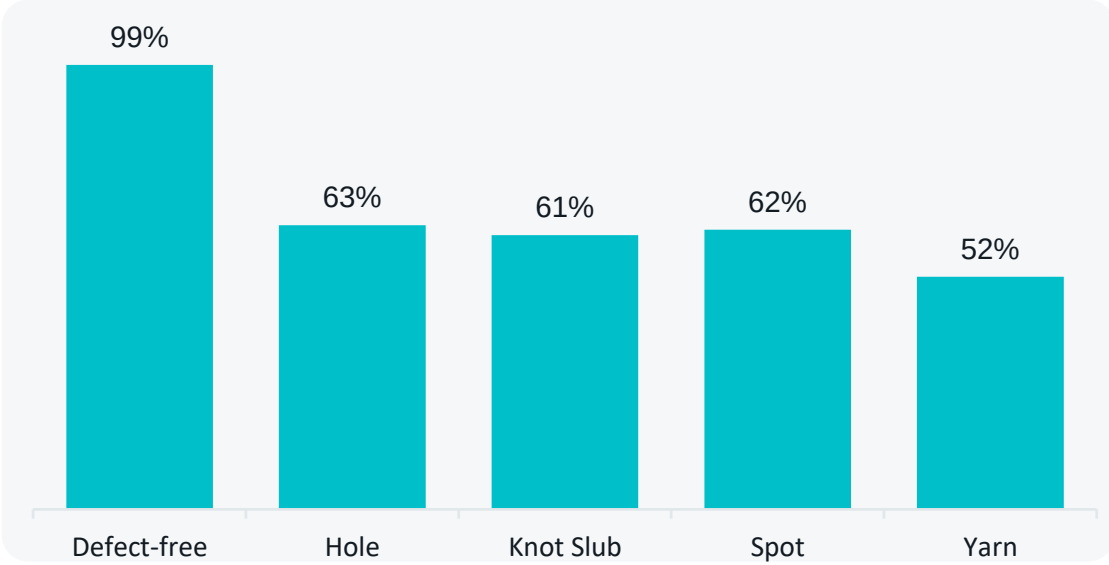


Figure 10. YOLO26s per class mAP@50

Model Comparison

Result Comparison Between Models

Model	Best Epoch	Precision (%)	Recall (%)	F1 Score (%)	mAP@50 (%)	mAP@50:95 (%)
YOLOv8s	**372**	**96.27**	**94.28**	**95.27**	**98.56**	**75.83**
YOLOv9s	377	93.71	89.86	91.75	95.73	66.69
YOLOv11s	381	93.07	89.58	91.29	96.26	71.24
YOLO26s	349	94.02	93.48	93.75	97.95	70.75

Table. Overall Performance Comparison of Models

Model Comparison

Result Comparison Between Models

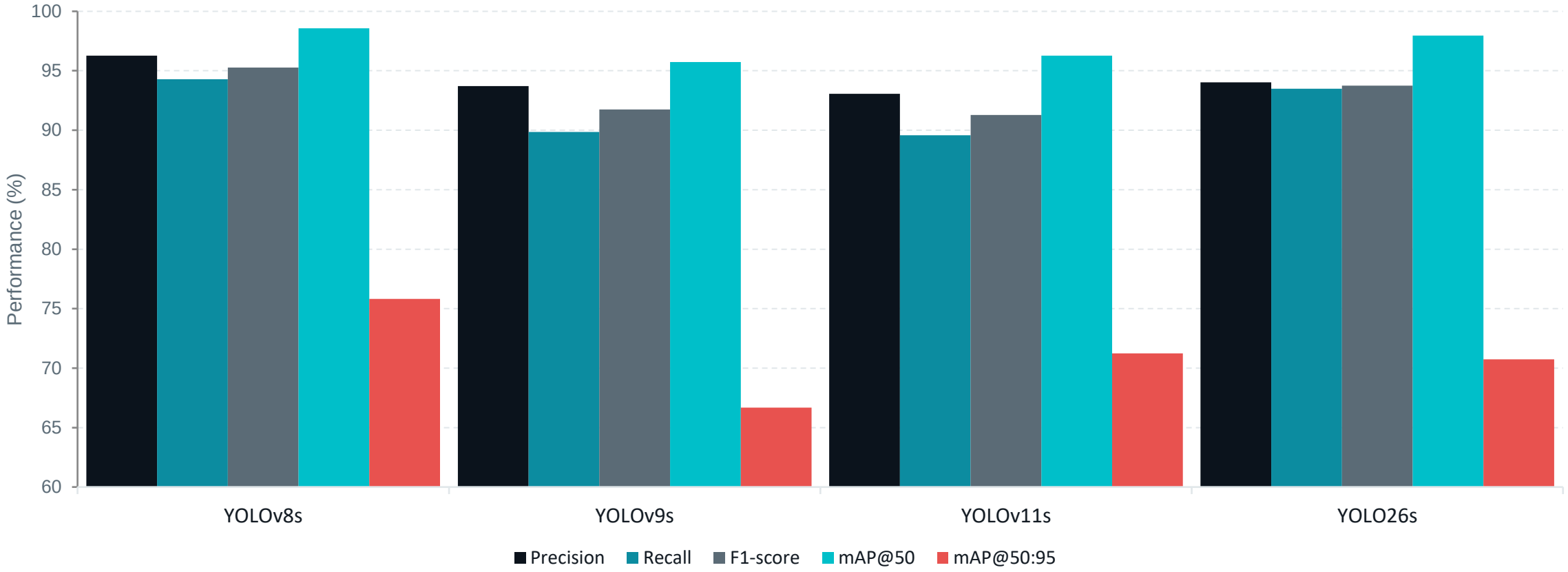


Figure. Model Performance Comparison

Where detection succeeds — and struggles

EASIER TO DETECT

Hole and spot defects have clear visual boundaries against the fabric — consistently high per-class mAP (0.63–0.73) across all four models.

HARDER TO DETECT

Yarn and knot-slub defects are thin, irregular, and visually close to the fabric's own texture — the lowest-scoring class in every model (as low as 0.43).

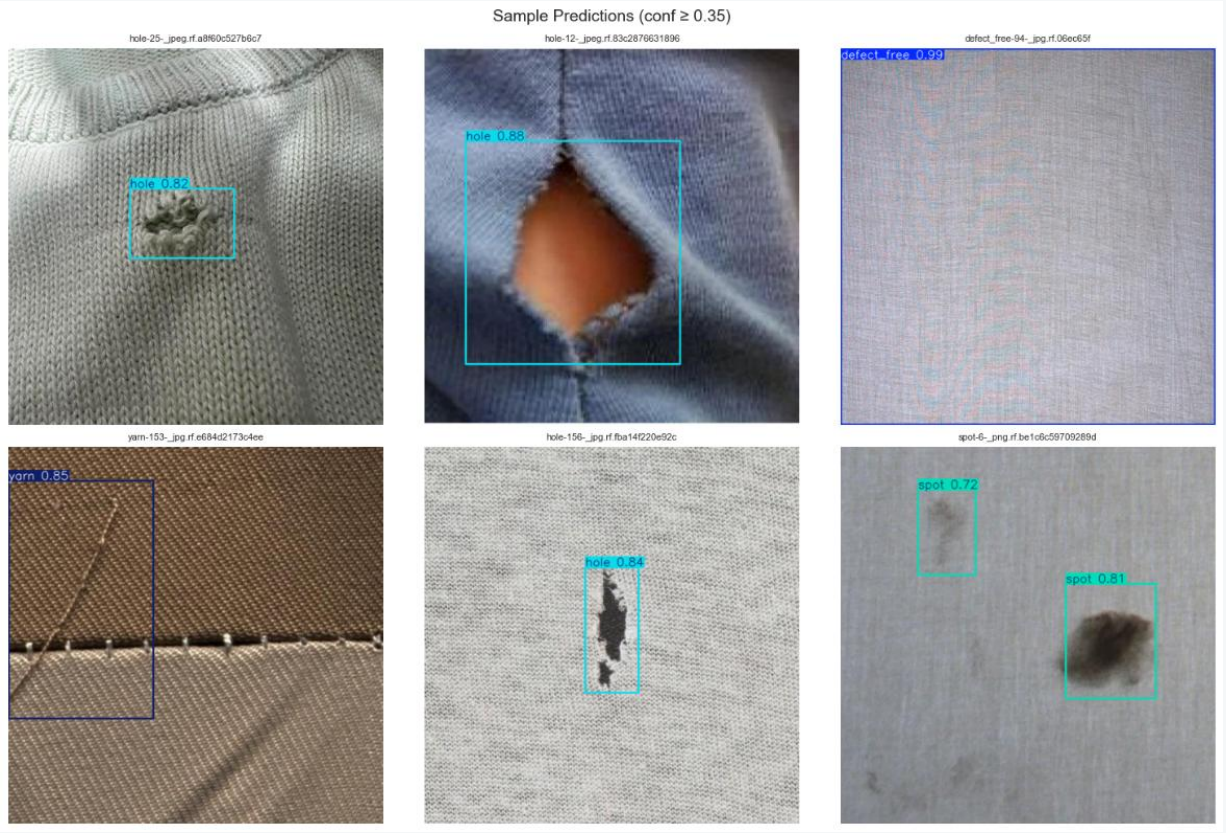


Figure11 — YOLOv8s sample predictions (confidence ≥ 0.35)

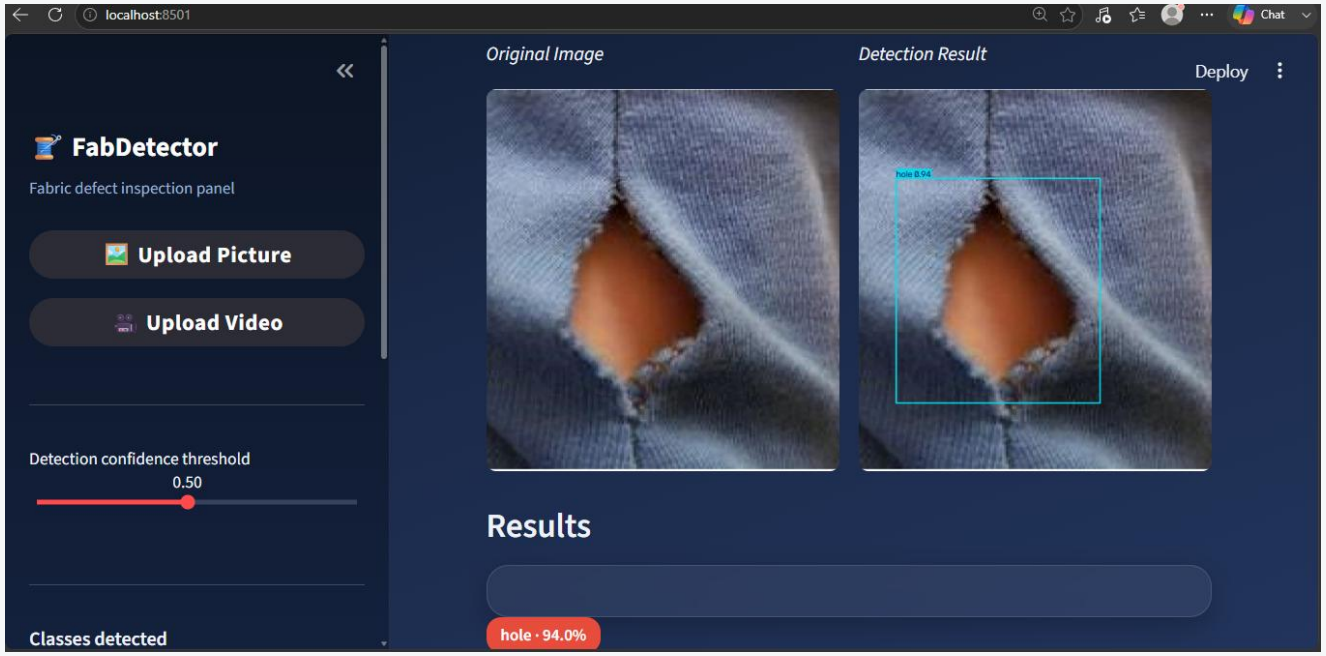
Web Application





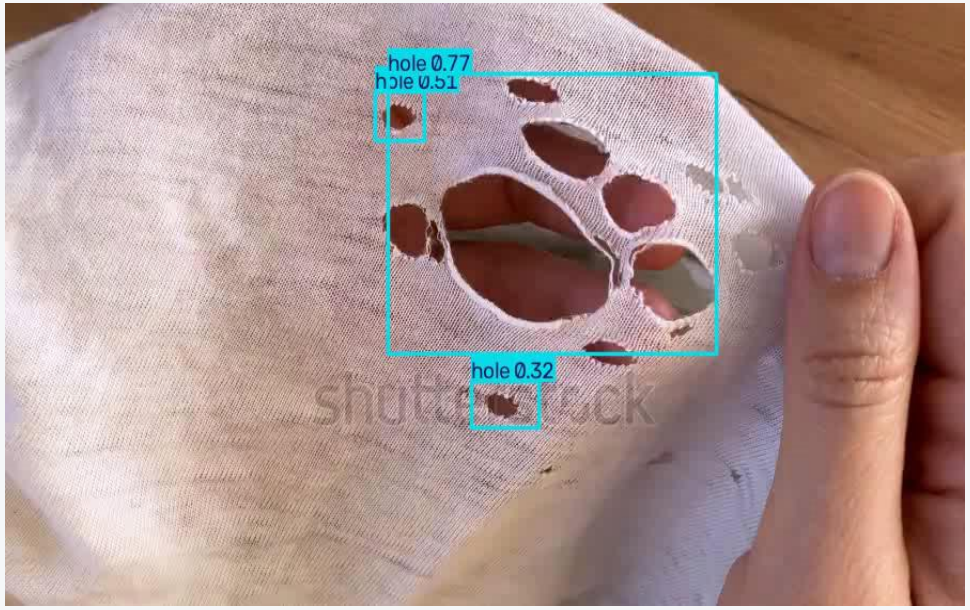
Feature	Description
Framework	Streamlit
Model	YOLOv8s (Best Performing Model)
Input	Image (.jpg, .png, .jpeg) and Video (.mp4, .avi, .mov)
Confidence Slider	Adjustable threshold (0.0–1.0)
Detection Output	Bounding boxes, class labels, confidence scores
Result Summary	Number of detected defects by category
Performance	Real-time inference
User Interface	Web-based, interactive, and lightweight

Live demo — image & video detection



Original vs. detection result — hole flagged at 94% confidence

VIDEO INFERENCE

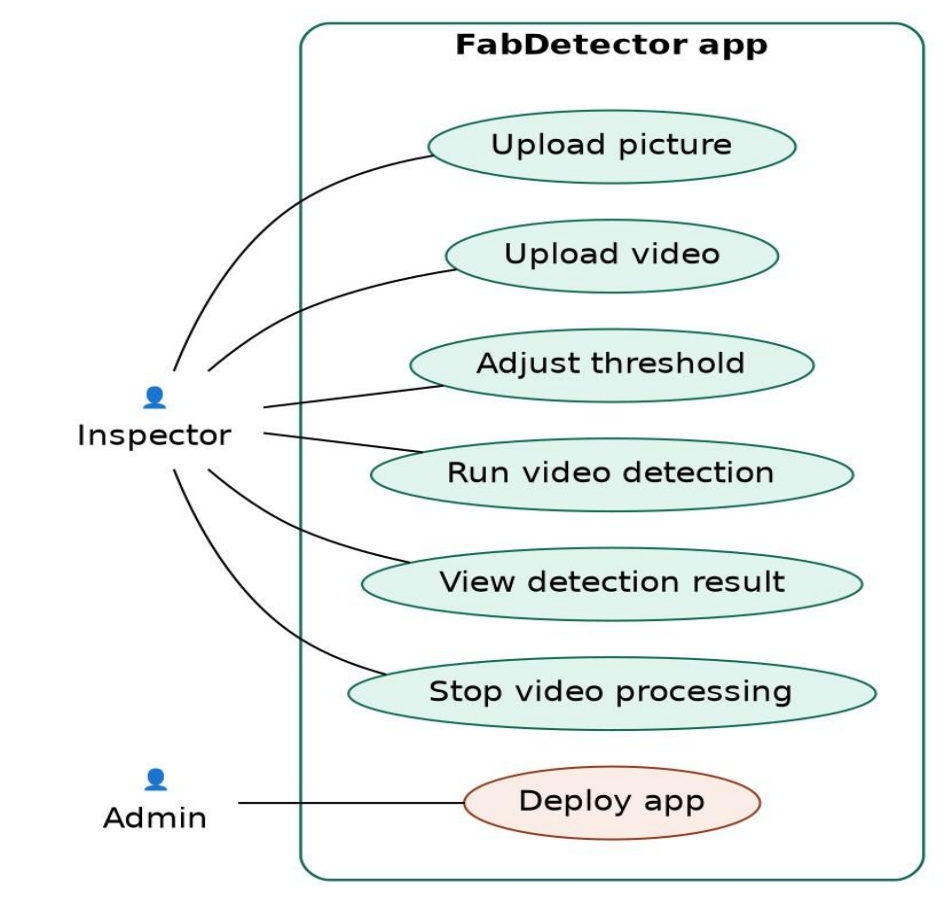


Live confidence-threshold controlled detection on video, run frame by frame.

The confidence-threshold slider lets an inspector tune sensitivity per fabric type — higher for noisy textures, lower for subtle defects.

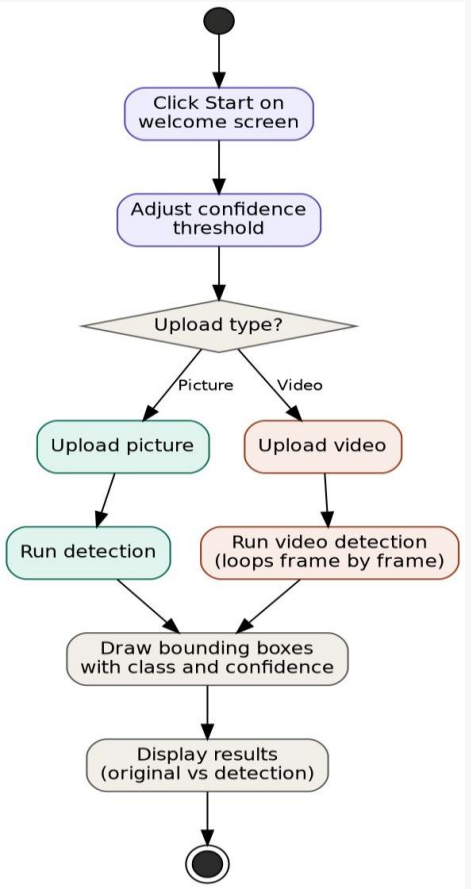
FabDetector — UML diagrams

USE CASE DIAGRAM



ACTIVITY DIAGRAM

Inspector uploads pictures or video, adjusts the confidence threshold, runs detection, and views results. Admin deploys the app.



Key findings

1 YOLOv8s is the most balanced

Stable convergence, minimal overfitting, and the best score on every meric evaluated.

2 YOLO26s trains fastest

Reaches competitive accuracy in only 349 epochs — attractive for constrained deployments.

3 Fine-textured defects remain hard

Yarn and knot-slub are the consistent weak point across all four architectures.

4 Errors are mostly misses, not confusion

Once a defect is detected, the model almost always assigns the correct class.

Limitations

- Only five defect categories — doesn't represent the full range seen in industrial production.
- Yarn and knot-slub defects visually resemble normal fabric texture, limiting detection accuracy.
- Images were captured under controlled conditions; real factories vary in lighting and background noise.
- Only lightweight YOLO architectures were tested — RT-DETR, EfficientDet, and transformer-based detectors were out of scope.
- Efficiency was assessed via training convergence, not real-time benchmarking on embedded/edge hardware.

These constraints define clear next steps rather than weaknesses in the core approach — see Future Work.

Conclusion & future work

CONCLUSION

- YOLOv8s delivers the best overall performance (96.27% precision, 98.56% mAP@50) among all four lightweight architectures tested.
- The framework offers accurate, consistent, real-time defect detection — a practical alternative to manual inspection.
- FabDetector demonstrates that this pipeline is deployable as a working, usable industrial quality-control tool.

FUTURE WORK

- Expand the dataset — more fabric types, lighting conditions, and defect categories.
- Explore attention mechanisms and transformer-based feature extraction for subtle defects.
- Deploy on edge AI hardware (e.g. NVIDIA Jetson) for low-latency, on-line inspection.
- Integrate with industrial cameras, conveyor systems, and MES for full automation.

THANK YOU ALL!

Questions & Discussion-